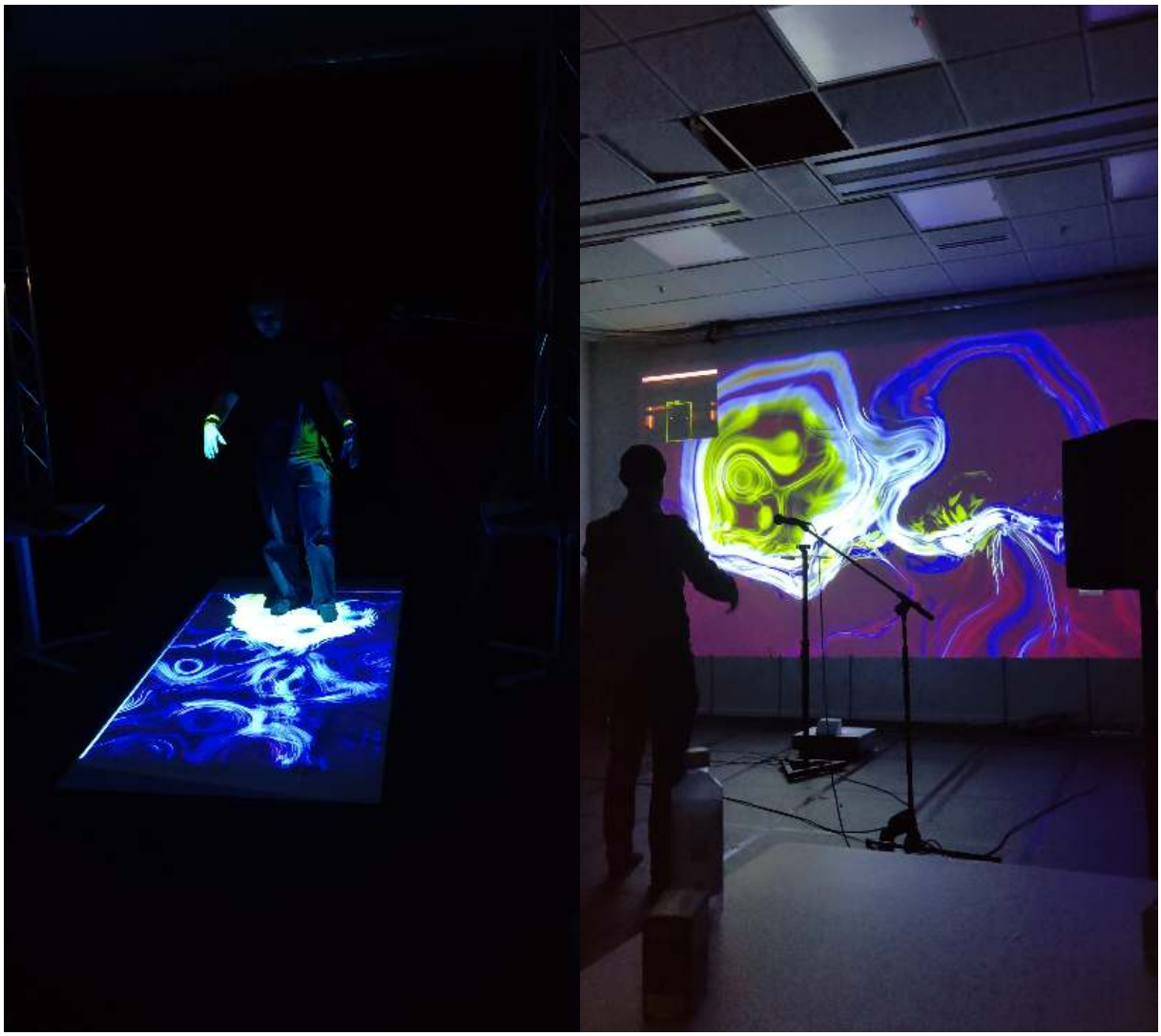
AHCI Exam Project

Frederik Laust
Kirchhoff Nørregaard
Aalborg University
Study no.: 20233920
fnorre23@aau.student.dk

Søren Pugerup Lasskogen
Aalborg University
Study no.: 20234281
slassk23@aau.student.dk

Mathias Jaqué
Aalborg University
Study no.: 20233906
mjaque23@aau.student.dk



1 Intro

As we last semester finished working on a framework for creating interactive art installations, an idea about walking on an interactive audiovisual installation popped into our heads. It would work great with the curriculum, and allow us to further develop some of the skills required for creating interactive installations.

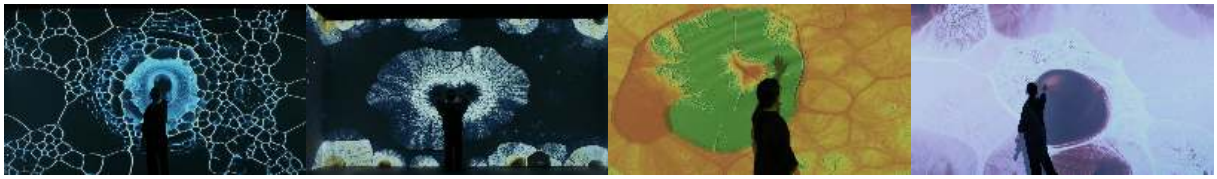
The system of this mini-project is aiming to create an interactive audiovisual installation, that uses multiple modalities, both as input and output. By using object detection models to track the user, we would be able to send the data between computers via Open Sound Control (OSC), into audio and visual programs like TouchDesigner and Ableton Live. In short, the system is trying to create an interactive experience for users, beautifully intertwining human-computer interactions.

2 Related Work

The following section with look at related work that exists working within the same medias:

2.1 The Improvisation of Water

This project is a realtime art installation, that uses detection, presumably through Kinect cameras. Using the data to drive the visual interaction in a 360° immersive projection room [1].



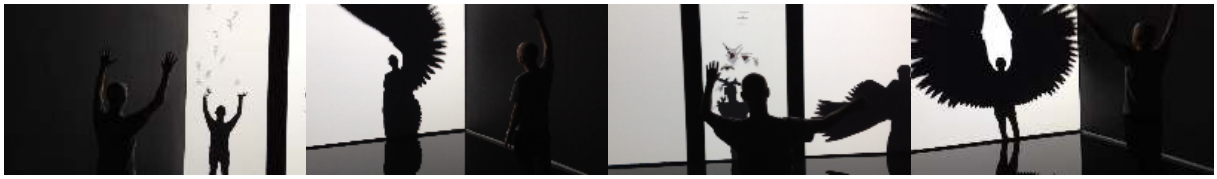
2.2 teamLab: Life

This project is also a realtime art installation, that uses lidar to track people in the room. The visuals change, depending on where people are standing and walking around in the room [2].



2.3 The Treachery of Sanctuary

This project uses a Kinect and Lidar to track users and map them on a projection, turning them into birds [3].



2.4 teamLab: Koi and People

This project uses lidar to track people, which continuously change the artwork. The visuals are projected on the surface of a pool of water, where real-time generated koi fish are swimming around. As the fish collide with people or other fish, they explode into flowers [4].



3 Modalities

This project four multiple modalities; 2 input modalities, and 2 output modalities. For input, the installation employs an auditory input, with a Shure SM58 microphone, setup to capture the voice, or otherwise noisemaking (e.g. clapping) of the user. This provides a real-time interaction for the user, where, if they hit a certain noise-level, ripples will be sent from their bubble on the floor projection. As it is only accounting for noise-level, both a clap, or the person yelling, or just generally getting close to the mic and doing something, triggers the effect. The second input is kinesic, as the user's position is tracked, through a web camera feeding the input to a Python program running the YOLOv11 AI object detection model. The user's position provide 2 real-time interactions. It both serves as the position of their bubble in the projection, seemingly following the user around, and then their position serves as the x and y value for inputs to Ableton live controlling assignable parameters on 2 audio effects on a sample which are played into the room. This audio being played, is one of the 2 output modalities, the second being the visual projection, where a bubble is following the user around.

It could be argued that the user's position is not necessarily kinesic, as it doesn't exactly track the movement of the user explicitly, only their current position with no relation to the previous position. But the resulting output at the very least, provides a convincing illusion of movement tracking, and invites movement to explore the effects of their placement on the projection.

4 Architecture

The resulting system contains 3 main components:

1. A Python program which uses YOLOv11 AI from Ultralytics to track humans in real time.
2. A TouchDesigner patch, which receives the positional data from the YOLO tracking, and the auditory data from a microphone.
3. An Ableton Live session, which receives the positional data from the YOLO tracking, and uses the data to control parameters of audio effects.

See figure Figure 5 for a general overview of dataflow, and input and output modalities.

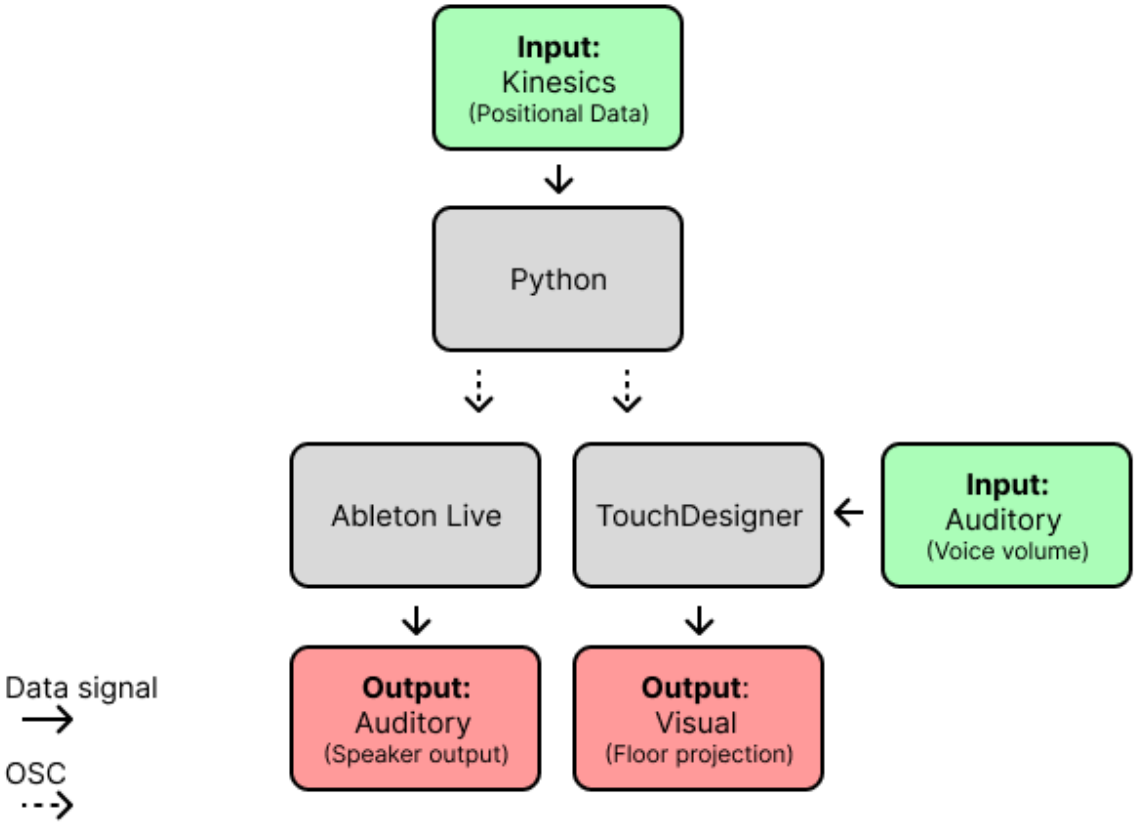


Figure 5: The data flow of the system

4.1 Physical setup

Please refer to Figure 6.

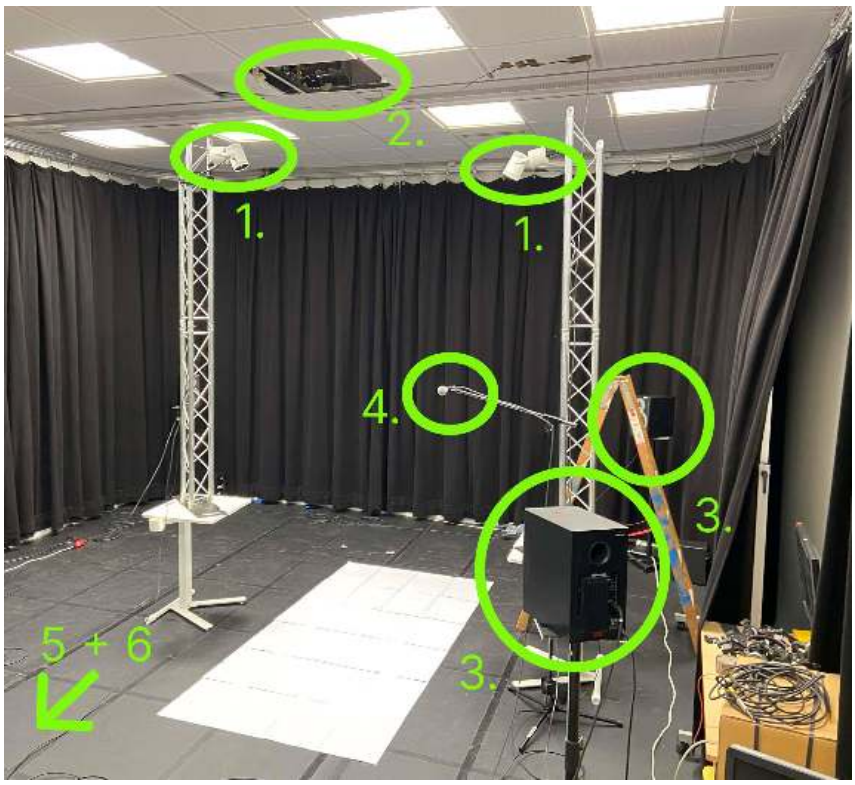


Figure 6: The installation's physical setup

The installation's setup consisted of the following components:

1. 2 x Samsung Freestyle mini projectors
2. 1 Logitech webcam
3. 2 x Dynaudio BM5mk2 studio monitors
4. 1 Shure SM58 microphone
5. 1 computer running a Python program and a TouchDesigner patch
6. 1 computer running an Ableton Live session

As the purpose of the installation is to watch and interact with the projection, specifically beneath your feet, one's shadow might interfere with this. The purpose of having 2 projectors, is to combat these potential shadows. Overlapping the 2 projectors' output, reduces the shadows, as they can fill in for each other, where a shadow might drop. The overlapping projections where set up manually.

The webcam is setup in the ceiling on a small laser cut mount, and connected via a USB-A cable extender. By suspending it directly above, we are able simplify the code, as we do not need to account for any 3D to 2D coordinates conversion. The webcam captures the 2D perspective directly, and the YOLO model returns the 2D values of the user's position as desired.

The 2 computers are connected via a local hotspot, and are communicating through OSC.

4.2 Python program

The Python program is responsible for capturing, manipulating, and sending the positional data as OSC messages to the TouchDesigner patch, and Ableton Live session. It uses the following imports to achieve this:

```
import cv2
import argparse
```

```
from pythonosc import udp_client
from ultralytics import YOLO
```

- `cv2`: OpenCV for Python. Provides functionality for capturing, displaying, and drawing on webcam footage. Additionally, it makes it possible to use live webcam footage as input for our AI tracking.
- `argparse`: Provides parsing of command-line arguments.
- `python-osc`: Provides simple UDP-client functionality, capable of sending OSC messages.
- `ultralytics`: Provides AI object and pose detection.

Before continuing, it should be noted that the original vision for the project, was that up to 3 people should be able to to interact with the experience at once, which is why the code accommodates for a flexible number of participants. A decision was made during the final stages to only allow 1 person in the experience at a time, due to a tracking error with the YOLO model, causing ID's to be switched, which made for a less than ideal experience. Ultimately, the source code still reflects the original intent.

4.2.1 Main loop

The core loop of the program is executed through OpenCV's `while True` convention, which reads the camera input every frame, so that we are able to manipulate the data to our program needs, then sending the data off as OSC messages to TouchDesigner and Ableton Live.

See the following code snippet for a general overview of the program flow:

```
# ... Setup ... #

videoCap = cv2.VideoCapture(args.cam)

# Main loop
while True:

    ret, frame = videoCap.read() # Reading webcam frame

    if not ret:
        print("Camera error. Closing...")
        break

    # AI object detection on the frame
    results = yolo.track(frame, classes=0, persist=True, tracker="bytetrack.yaml")

    for result in results:
        # Data manipulations
        # Sending data to TouchDesigner and Ableton Live
        # Drawing results on frame

    # Display frame
    cv2.imshow("Frame", frame)

    # q to quit
    key = cv2.waitKey(3) & 0xFF
    if key == ord('q'):
        break

# Releasing resources on program termination
cv2.destroyAllWindows()
videoCap.release()
```

4.2.2 YOLO AI tracking

The program uses the YOLOv11 AI provided by Ultralytics [5], specifically the yolo11n model, which is their smallest model. We chose the smallest model in terms of parameters, in order to improve on frame rate, as a bigger model would require more resources, and thereby slow down the detection time per frame. According to Ultralytics own page, choosing the second smallest model, would almost double the detection time per frame. On top of this, the computer running this program, is also the computer which will be running and rendering the TouchDesigner patch.

The tracking via Python is implemented with a single line of code, where keyword arguments are passed to specify settings for the tracking. We use the following settings:

- `classes`: Determines what the model searches for. 0 is for humans.
- `persist`: Persists tracks between frames.
- `tracker`: Determines the tracker which are used. BYTEtrack allows to keep track of lost objects between some frames, helping with possible occlusion issues.

In the program the tracking is implemented as such, using the `.track()` method:

```
# Help with settings for the track() method https://medium.com/@beam_villa/object-tracking-made-easy-with-yolov11-bytetrack-73aac16a9f4a

results = yolo.track(frame, classes=0, persist=True, tracker="bytetrack.yaml")
```

The `results` object then holds all results found in the frame, such as detected humans, which contains a ton of data describing the detected object(s). For the purposes of this project, only the bounding box of the object, the confidence score, and ID were accessed and used, however for possible future use cases, skeletal tracking were also setup. The position of the user, are not determined by any one quality of them such as certain limb, or facial feature, but by the center of their bounding box. When tracking humans from above, the system was found to be quite unreliable, especially when attempting to track skeletal data. In an attempt to track a user's nose, the point would jump quite rapidly between frames, producing rough results, hence the decision to track the bounding box.

The positional data received were manipulated, to accommodate both TouchDesigner's and Ableton Live's needs, as each program needed different ranges of numbers. Specifically, TouchDesigner requires the numbers to be between −1 and 1, and Ableton Live requires the numbers to be between 1 and 100. In practice, the `x` and `y` data of the bounding box middle point were calculated, and then two tuples were declared, with the calculated values for Live and TouchDesigner separately.

In the program, it is implemented as such:

```
# Retrieving bounding box data
x1, y1, x2, y2 = map(int, box.xyxy[0])

# Calculating coordinates for middle of bounding box
point_x = (x2-x1) * 0.5 + x1
point_y = (y2-y1) * 0.5 + y1

# Converting values to between -1 and 1 for TouchDesigner
point_to_TD = (
    1 - (point_x / w) * 2, # w is width of the camera capture
    1 - (point_y / h) * 2  # h is height of the camera capture
)

# Converting values to between 1 and 100 for Ableton live
point_to_Live = (
```

```
    int(point_x / w * 100), # w is width of the camera capture
    int(point_y / h * 100)  # h is height of the camera capture
)
```

4.2.3 OSC messaging with Python

Once the data has been prepared for send-off, it has to be formatted to an OSC message. To achieve this, the `python-osc` library provides a simple UDP client, which are able to send OSC messages via UDP. As TouchDesigner and Ableton Live, are running on two separate computers, we need to setup a UDP client for each.

The messages are also formatted differently per program. In the initial development phase, there was a focus on flexibility, meaning that the OSC message were designed to accommodate any scenario and data wanted by the TouchDesigner patch, and as such, the messages contained an address of the person (e.g. "/ID1"), and then the message would contain multiple arguments. In this fashion, it would be possible to put an arbitrary number of arguments into 1 message, which the TouchDesigner patch could retrieve and dissect as needed. It would only need to know the order of the arguments, and what they represent. Once Ableton Live was added to the mix, it was found that it was too difficult to split up the message, so instead of one address with multiple arguments, it would need one address per relevant datapoint, and then only a single argument (e.g. "/ID1_x value").

At this point in development, it was decided that only the middle point of the bounding box would be needed, and that the experience would only be designed for a single person. This decision was made based on multiple factors, though this factored in.

In the program, it is implemented as such:

```
# OSC client til TD
td_client = udp_client.SimpleUDPClient(args.ipTD, args.portTD)

# OSC client til Live
ableton_client = udp_client.SimpleUDPClient(args.ipLive, args.portLive)

# Main loop
while True:

  # Tracking and data manipulation logic #

  # Collects and formats all data for TouchDesigner
  message_to_TD = []
  message_to_TD.append(f"{point_to_TD[0]:.2}") # Formats float to 2 decimal points
  message_to_TD.append(f"{point_to_TD[1]:.2}") # Formats float to 2 decimal points

  # Sends message to TouchDesigner
  td_client.send_message(f"/ID{person_ID}", message_to_TD) # The person ID is also a
part of the result object, which are retreived at an earlier point

  # Send message to Live
  ableton_client.send_message(f"/ID{person_ID}_x", point_to_Live[0])
  ableton_client.send_message(f"/ID{person_ID}_y", point_to_Live[1])
```

4.3 TouchDesigner

The visual engine was used to drive the visual interactive elements. It received positional OSC data from Python. Furthermore, microphone audio was also used as input.

4.3.1 The Main Engine

The main engine of the TouchDesigner, is a patch that handles all of the main input and output.

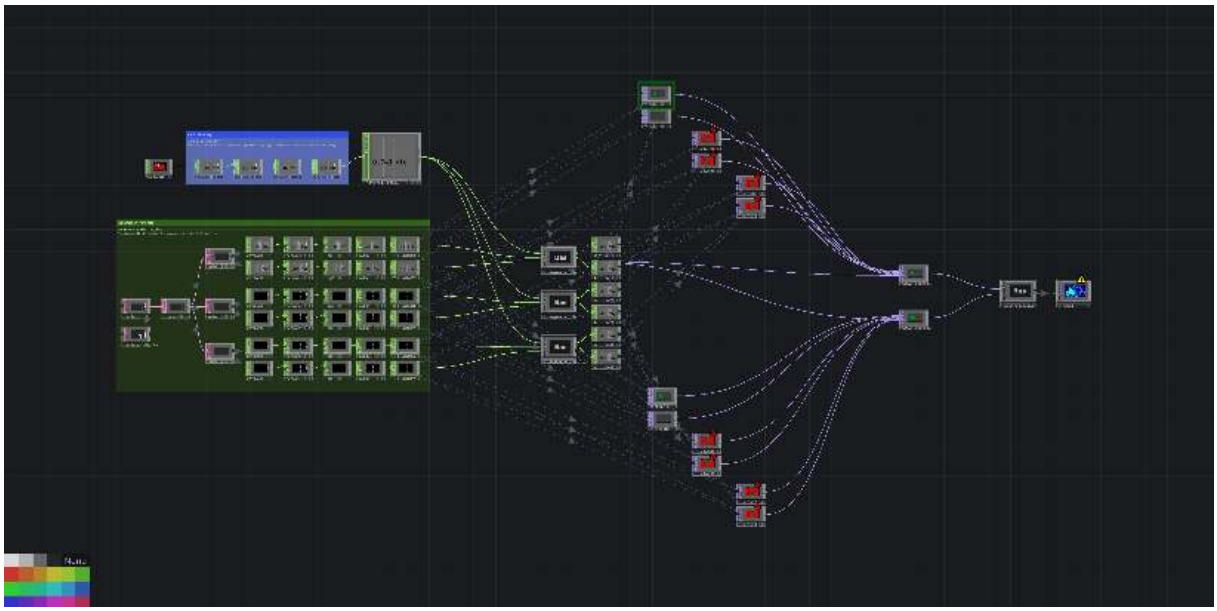


Figure 7: Overview of TouchDesigner Patch

4.3.1.1 Input

Looking at the overview of the main TouchDesigner patch Figure 7. The input is seen on the left side of the patch. One part is for taking a microphone signal as input, filtering the signal through a One euro filter, a math operator and expression operator, to alter the range and amount of decimals. The other input, is all the OSC positional data sent from Python. This data is parsed into multiple ID's and channels, to keep track of the users and their X and Y positions.

4.3.1.2 Output

In the main patch the output is simply sending a visual output feeding to the projector.

4.3.1.3 Interaction

The positional data and microphone signal is sent into a sub-patch seen in Figure 8. In this sub-patch mainly the microphone signal drive the circumference of the circle following the user. Furthermore, there is an event operator that keeps track of the user ID's and once a certain signal threshold is met, it will expand the circle continuously giving a pulse effect.

The positional data is sent too all circles, both in the sub-patch and outside so the circles always follow user.

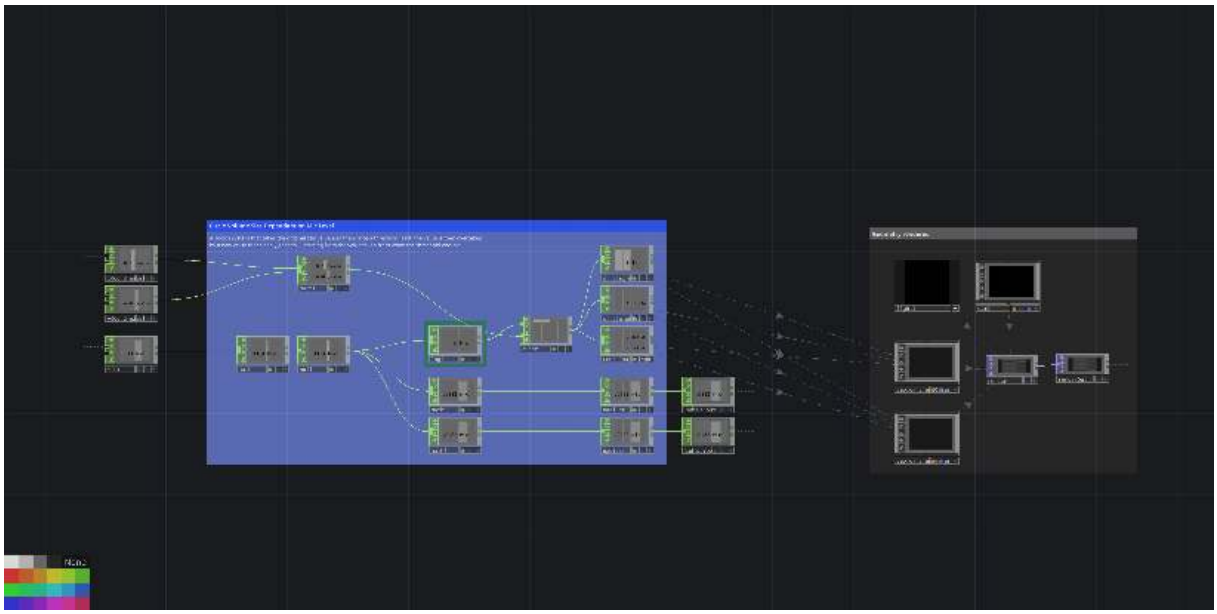


Figure 8: Overview of TouchDesigner Patch

4.3.1.4 Post Processing

In the last sub-patch is where all the visual effects are processed and post-processed. The sub-patch is created with two noise operators that is controlled with three LFO operators, giving it an oscillating effect. The output of the noise operators goes through various Texture Operators like - Displace, Edge, Multiply, Bloom, Composition giving the final visual look.

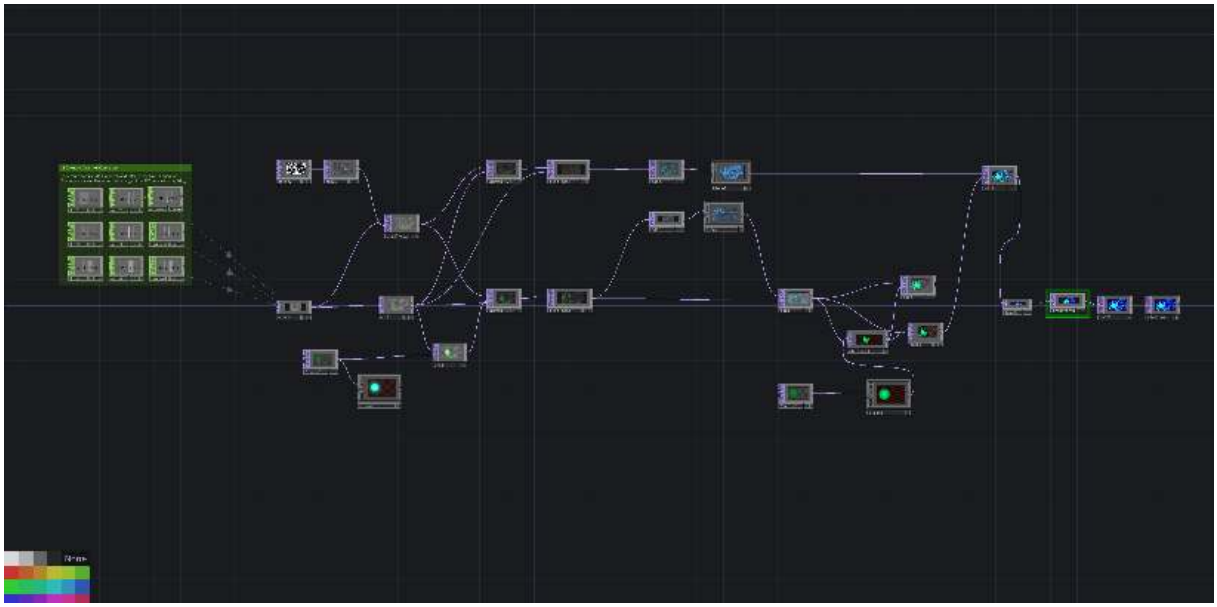


Figure 9: Overview of TouchDesigner Patch

4.4 Ableton Live

The general idea was to take incoming OSC values and map them to audio effects and plugins using Ableton Live's built in in mapping function where anything can be mapped to everything. Then output the sound via four Dynaudio speakers, setup around the user to give a surrounding soundscape by panning the speakers in pairs of two ultimately giving the system four channels of output and a wider control of mapping and routing opportunities. However because of bleed into the SM58 mic and feedback we ended up only using two speakers in a stereo setup.

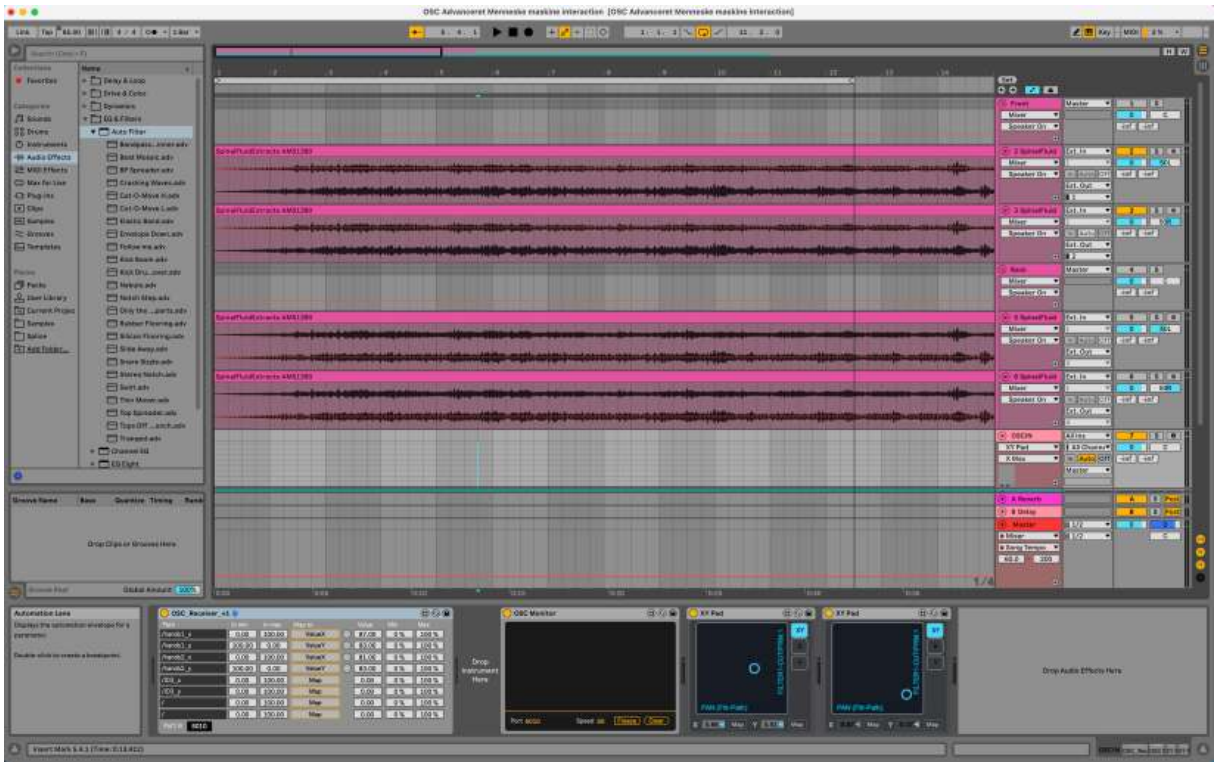


Figure 10: Ableton Live setup

4.4.1 OSC Receiver

The setup in Ableton live was rather simple but quite effective. The OSC messages received from the YOLO model was received using the OSCReciever v1 by ctrlz [6] which reads the messages and path on the incoming UDP port and makes it possible to map the data to any plugin or effect used in the project. It is possible to manipulate the incoming values to clamp them between a min and max value only reading valid data and filter out potential extremes, these can then be referenced to a percentage of a mapped knob or effect giving more customization if needed. We went with 0-100 both in value and percentage, controlling the entire range of a given effect.

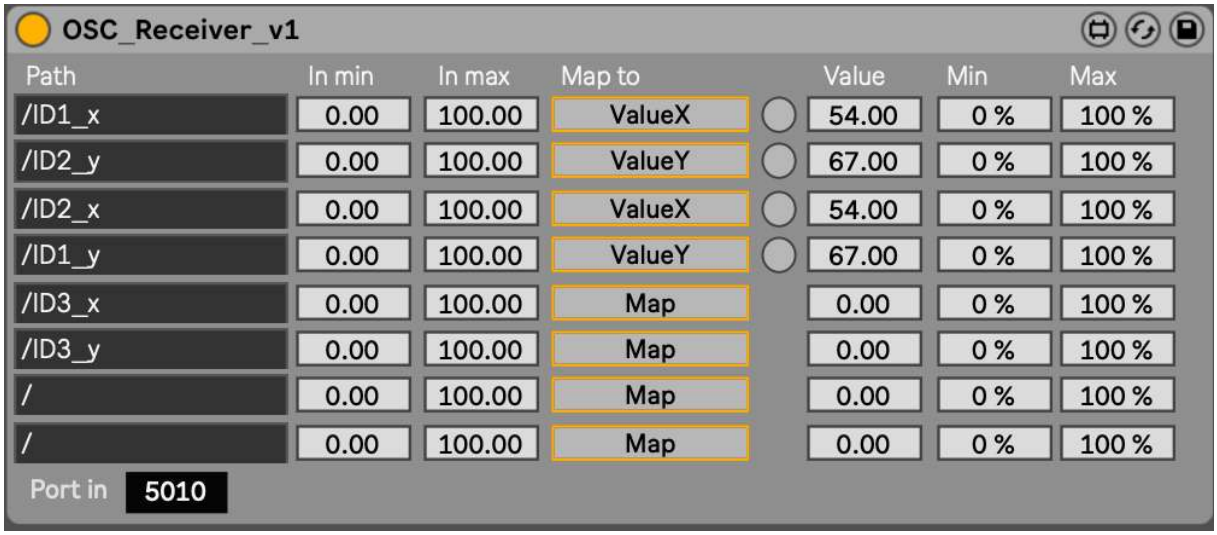


Figure 11: OSC Receiver patch

4.4.2 Sample and Effects

In our case we used a sample of a underwater recording that sonically matched the visuals from TouchDesigner. The original idea was to map each person with a track of a sample with the X,Y

values resembling a chaos pad, setup with two effects per person. Because of the previous mentioned problems with the YOLO model we simplified it to one persons X,Y values but mapped it twice to two XY pads, receiving the same data but then controlled two effects with a total of four parameters.



Figure 12: XY pads with mapped effect controls

We used an EQ with the X value controlling the filter frequency and y controlling the resonance and a tremolo ring modulator where X controlled the coarse and Y controlled the LFO Rate.

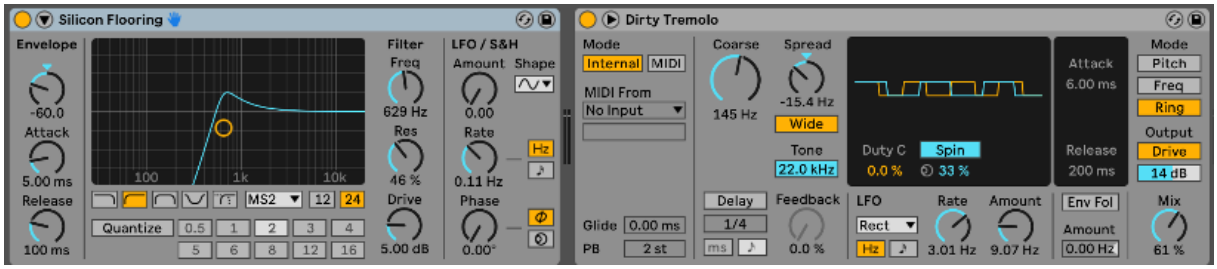


Figure 13: Effect Rack EQ and Tremolo in ring modulation mode

5 Conclusion and future work

The project suffered one major issue; the YOLO model being dependent on a lit, and mostly unobscured subject in order to produce reliable object detection. This limitation ultimately resulted in the down-scaling of the project, going from a 3-person experience to a 1-person experience, and even with the downscaling, still produced unpleasant jittery projections. The experience could potentially be improved by replacing the webcam with a lidar camera instead, which could provide better footage for the YOLO model to analyze. By having a cleaner input, it should also be easy to increase the amount of people who can interact with the installation, as this was setup before the tracking limitation was found. Whether the YOLO model is able to perform its detection process on lidar footage, is however, unknown. In a general optimization sense, the Python code could be directly implemented within TouchDesigner, as it allows for custom modules coded with Python. This could reduce latency between the object detection and the visual projection, as it would no longer have to be sent via UDP.

Some other improvements, could be to replace the current microphone setup, with cardioid characteristic lavalier microphones, allowing for individual audio input while also reducing bleed. This could expand on the interactivity of each person's own bubble, instead of a collective output of

the audio input. Furthermore, incorporating physiological measures as input, could also help expand on the individual interactivity of each person's bubble. Perhaps a heart-rate monitor, allowing one's bubble to pulsate alongside one's heartbeat. Perhaps electro-dermal activity could be monitored, measuring one's sweat, which could change the color of one's bubble. These types of input would provide a more individualized experience, and perhaps an awkward first date attraction, where one's nervousness would literally be visually represented.

Despite room for improvement and the unfortunate need for down-scaling, we believe that the experience incorporated all the elements we set out to. The experience still worked as originally intended, but it just did not quite live up to our expectations, since the projectors were too weak to provide a clear image with lights on, and the object detection performed poorly when the lights were off to achieve a better projection. This tradeoff, made it so the experience felt weak, in one way or another.

A v2 of this installation was set up after the original, changing the installation from a floor projection, to a wall projection, where the user's hands are tracked, instead of their body's position. Each hand represented a bubble in the projection, and this increase in points to track, led to a more expansive auditory experience, considering more parameters could be altered. It also allowed for a different projector, - as it did not have to be mounted high up - leading to a much bigger and brighter image. This performed remarkably better as well, as the camera would get a clear image of the people to track, with no projection hitting them, and the light from the installation was so bright it lit up the persons in the video feed, enough for the object detection to perform well. While this version performed better, the original intent was to create a full body experience, which v2 did not quite achieve, but it was still very fun, and entertaining to play around with.

Bibliography

[1] "The improvisation of water (2023) with Touchdesigner." Accessed: Dec. 04, 2025. [Online]. Available: https://www.are.na/block/31346174

[2] "teamLab: LIFE." Accessed: Dec. 04, 2025. [Online]. Available: https://www.teamlab.art/e/ddp/

[3] "The Treachery of Sanctuary - CHRIS MILK." Accessed: Dec. 04, 2025. [Online]. Available: http://milk.co/treachery

[4] "Drawing on the Water Surface Created by the Dance of Koi and People - Infinity." Accessed: Dec. 04, 2025. [Online]. Available: https://www.teamlab.art/w/koi_and_people/

[5] Ultralytics, "YOLO11." [Online]. Available: https://docs.ultralytics.com/models/yolo11/

[6] ctrlz, "OSC Receiver for Max for Live." [Online]. Available: https://maxforlive.com/library/device/7752/osc-receiver-osc-in